Respuestas - Pantallazo (Pruebas)

Johan Alexander Peña Ortiz · Ingeniería de Software II · Universidad Nacional de Colombia · 2026-2

Preguntas Material_Estudio.

S — SRP: Single Responsibility Principle.

Question 1. ***If I describe this class in one sentence, does the word "and" show up?***

R: Sí, en el "Before", "Car" se mueve y se imprime en pantalla y guarda un log en archivo". Tras la refactorización, Car solo "actualiza su posición", y RaceLogger sólo "registra y guarda el historial de la carrera".

Question 2. ***Who would ask me to change this class — and is it more than one kind of person or concern?***

R: La clase la podría cambiar quien define las reglas de movimiento, quien decide el formato de pantalla (UI/presentación) y quien decide cómo se persisten los datos (formato del log, archivo o base de datos); aunque creo que eso violaría el SRP.

O — OCP: Open/Closed Principle

Question 1. ***To support a new case, do I edit an existing class, or add a new one?***

R: Se debería de añadir una nueva clase que extienda la funcionalidad, sin modificar las clases que ya existen y funcionan correctamente.

Question 2. ***Does the code that runs the race (Track.run) need to know each vehicle type by name?***

R: No, solo necesita que cada objeto cumpla el contrato de Vehicle (llamar move()). Gracias al polimorfismo, Track.run funciona igual con Car, Motorcycle o cualquier vehículo futuro.

L — LSP: Liskov Substitution Principle

Question 1. ***What does the base class promise callers, implicitly or explicitly?***

R: Según el ejemplo, Vehicle.move() promete dos cosas: la posición nunca disminuye y nunca lanza excepciones.

Question 2. ***Could I swap this subclass in anywhere the base class is expected, and have nothing else notice?***

R: Sí, para cumplir LSP, cualquier subclase (como UnreliableCar) debe ser capaz de sustituir a su clase base (Vehicle) sin alterar la corrección del programa ni romper las expectativas del código que consume la abstracción base.

I — ISP: Interface Segregation Principle

Question 1. ***Does every implementer of this interface have a real, meaningful implementation for every one of its methods?***

R: No en el “Before”; en este, Bicycle está obligada a implementar fly() y refuel(), que no le corresponden, y tiene que lanzar NotImplementedError, lo cual es un code smell. En el “After”, Bicycle implementa solo Movable y Pedalable, y todos sus métodos son reales.

Question 2. ***If I split this interface in two, would any class need to implement both?***

R: Sí, Bicycle implementa Movable y Pedalable. Eso no es un problema, es la ventaja de las interfaces pequeñas: cada clase combina solo las que le aplican, y quien necesita solo Movable no depende de pedal_harder.

D — DIP: Dependency Inversion Principle

Question 1. ***Does this class construct its own collaborators, or receive them?***

R: En el “Before”, Race las construye (SportsCar("Flash"), DeliveryVan(...)), quedando acoplada a clases concretas. En el “After” las recibe por el constructor (racers, track), es decir, inyección de dependencias.

Question 2. ***Could I unit-test this class with a fake, in-memory version of its dependency, with no changes to the class itself?***

R: En el “Before” no. Siempre crea SportsCar y DeliveryVan reales. En el “After” sí. Se puede pasar un FakeRacer mínimo con move(), position y symbol, y Race funciona igual, porque depende de una abstracción y no de detalles concretos.

Preguntas Taller.

Exercise 1.

Why is it better for Car to know nothing about logging or printing? What would break first if a professor asked you to log results as JSON instead?

R:

a) Porque así Car solo cambia si cambia cómo se mueve. Imprimir y guardar son otros motivos de cambio, y mezclarlos hace que tocar uno pueda romper el movimiento.

b) Habría que editar Car.move(), que tenía la escritura al archivo mezclada con el movimiento. Ahora solo se cambia RaceLogger.save() (o se crea otro logger) y Car no se toca.

Exercise 2.

If you found yourself wanting to edit Vehicle or Track to finish this, what does that tell you about your design?

R: Que no es extensible, es decir, cada vehículo nuevo obligaría a modificar código que ya funcionaba. Aquí bastó con crear subclases con su propio move().

Exercise 3.

What would you have had to add to Track (extra try/except, position clamping...) if you had NOT fixed UnreliableCar? Why is fixing the subclass better?

R:

a) Un try/except alrededor de move() y código para corregir posiciones negativas. Track tendría que "desconfiar" de sus vehículos.

b) Se arregla en un solo lugar y el contrato (nunca retrocede, nunca falla) vale para todos los que usen Vehicle, no solo para Track.

Exercise 4.

Which interface(s) would a future Submarine or Airplane vehicle need? Does adding one require touching any existing vehicle class?

R:

a) El Airplane usaría Movable + Flyable (y Refuelable si carga combustible). El Submarine usaría Movable y quizá una nueva interfaz pequeña como Divable.

b) No. Solo se crea la clase nueva (y la interfaz nueva, si hace falta).

Exercise 5.

How does injecting the roster make Race easier to unit-test without a real terminal animation?

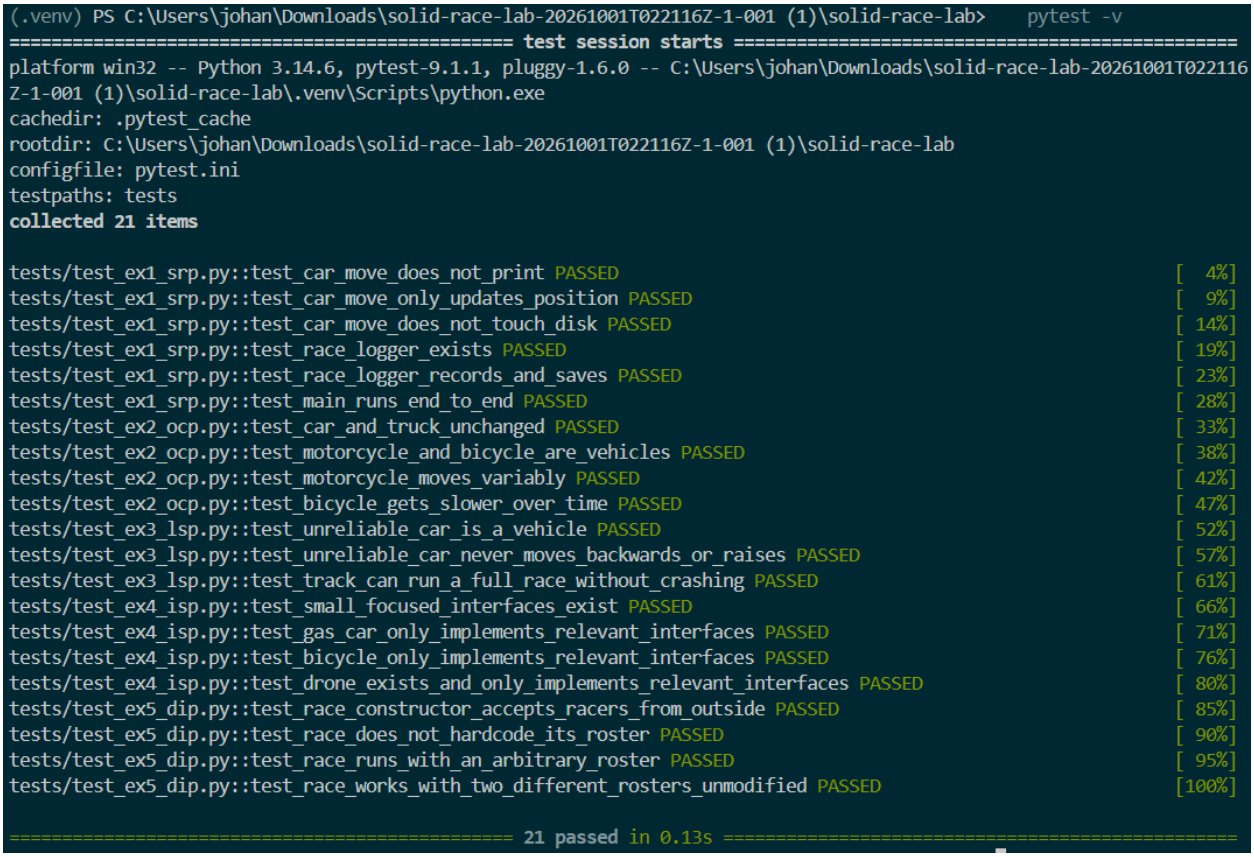
R: Se le pasan corredores falsos y sencillos (como _FakeRacer en los tests) y un Track sin animación. La prueba es rápida y no depende de terminal ni de vehículos reales.

Pregunta Final.

In your group, identify one place in your own course project (or a past project) where one of these five principles was being violated. What would you change?

R: No se me ocurre alguno en un proyecto propio, pero si se me ocurre un ejemplo, que sería una clase que consulta la base de datos y además envía correos (viola SRP). Se arregla separándola en una clase de acceso a datos y otra de notificaciones.

Pantallazo sobre las pruebas de los ejercicios.



```
(.venv) PS C:\Users\johan\Downloads\solid-race-lab-20261001T022116Z-1-001 (1)\solid-race-lab>    pytest -v
============================================= test session starts =============================================
platform win32 -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\johan\Downloads\solid-race-lab-20261001T022116
Z-1-001 (1)\solid-race-lab\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\johan\Downloads\solid-race-lab-20261001T022116Z-1-001 (1)\solid-race-lab
configfile: pytest.ini
testpaths: tests
collected 21 items

tests/test_ex1_srp.py::test_car_move_does_not_print PASSED                                          [  4%]
tests/test_ex1_srp.py::test_car_move_only_updates_position PASSED                                   [  9%]
tests/test_ex1_srp.py::test_car_move_does_not_touch_disk PASSED                                     [ 14%]
tests/test_ex1_srp.py::test_race_logger_exists PASSED                                               [ 19%]
tests/test_ex1_srp.py::test_race_logger_records_and_saves PASSED                                    [ 23%]
tests/test_ex1_srp.py::test_main_runs_end_to_end PASSED                                             [ 28%]
tests/test_ex2_ocp.py::test_car_and_truck_unchanged PASSED                                          [ 33%]
tests/test_ex2_ocp.py::test_motorcycle_and_bicycle_are_vehicles PASSED                              [ 38%]
tests/test_ex2_ocp.py::test_motorcycle_moves_variably PASSED                                        [ 42%]
tests/test_ex2_ocp.py::test_bicycle_gets_slower_over_time PASSED                                    [ 47%]
tests/test_ex3_lsp.py::test_unreliable_car_is_a_vehicle PASSED                                      [ 52%]
tests/test_ex3_lsp.py::test_unreliable_car_never_moves_backwards_or_raises PASSED                   [ 57%]
tests/test_ex3_lsp.py::test_track_can_run_a_full_race_without_crashing PASSED                       [ 61%]
tests/test_ex4_isp.py::test_small_focused_interfaces_exist PASSED                                   [ 66%]
tests/test_ex4_isp.py::test_gas_car_only_implements_relevant_interfaces PASSED                      [ 71%]
tests/test_ex4_isp.py::test_bicycle_only_implements_relevant_interfaces PASSED                      [ 76%]
tests/test_ex4_isp.py::test_drone_exists_and_only_implements_relevant_interfaces PASSED             [ 80%]
tests/test_ex5_dip.py::test_race_constructor_accepts_racers_from_outside PASSED                     [ 85%]
tests/test_ex5_dip.py::test_race_does_not_hardcode_its_roster PASSED                                [ 90%]
tests/test_ex5_dip.py::test_race_runs_with_an_arbitrary_roster PASSED                               [ 95%]
tests/test_ex5_dip.py::test_race_works_with_two_different_rosters_unmodified PASSED                 [100%]

============================================== 21 passed in 0.13s ==============================================
```